🎒 Guía Gráfica y Visual: El Problema de la Mochila 0/1

Esta guía gráfica está diseñada como material educativo complementario para la materia **Programación III**. Explica de forma estructurada, visual y paso a paso el funcionamiento de la Programación Dinámica aplicada al Problema de la Mochila 0/1, utilizando el ejemplo exacto de la presentación del curso.

---

1. Resumen de Objetos y Capacidad

Mochila: Capacidad Máxima P = 8

Regla 0/1: Los objetos NO se pueden cortar. Cada objeto se toma entero (1) o se deja (0).

Objeto (o)	Nombre / Descripción	Peso (peso[o])	Valor (valor[o])	Relación Valor/Peso
1	Objeto 1	1	10	10.0
2	Objeto 2	4	3	0.75
3	Objeto 3	5	5	1.0
4	Objeto 4	7	8	1.14

2. Matriz de Programación Dinámica (Paso a Paso)

Las filas representan los objetos disponibles acumulados y las columnas la capacidad actual de la mochila (de 0 a 8).

Objetos \ Capacidad (p)	0	1	2	3	4	5	6	7	8
Base (Sin objetos)	0	0	0	0	0	0	0	0	0
Objeto 1 (p=1, v=10)	0	**10**	10	10	10	10	10	10	10
Objeto 2 (p=4, v=3)	0	10	10	10	10	13	13	13	13
Objeto 3 (p=5, v=5)	0	10	10	10	10	13	13	13	15
Objeto 4 (p=7, v=8)	0	10	10	10	10	13	13	13	**18** 🎯

Leyenda de colores:

- Verde (18 🎯): Resultado final del algoritmo. Es el **valor máximo posible**.
- Naranja (10): Celdas clave identificadas durante el proceso de reconstrucción de la solución.

3. Diagrama Visual de Reconstrucción (Backtracking)

Para determinar **qué objetos forman el valor 18**, realizamos un recorrido desde el final (Fila 4, Capacidad 8) hacia el origen (Fila 0):

```
[INICIO] Celda M[4, 8] = 18
   |
   ├── ¿M[4, 8] (18) != M[3, 8] (15)? ———> ¡SÍ CAMBIÓ!
   |                                                  |
   |                                                  ├——▶ 🟢 SE INCLUYE EL OBJETO 4
(Peso: 7, Valor: 8)
   |                                                  └——▶ Capacidad restante: 8 - 7 = 1
   |
[PASO 2] Celda M[3, 1] = 10
   |
   ├── ¿M[3, 1] (10) == M[2, 1] (10)? ———> NO CAMBIÓ
   |                                                  |
   |                                                  └——▶ 🔴 SE DESPORTA EL OBJETO 3
   |
[PASO 3] Celda M[2, 1] = 10
   |
   ├── ¿M[2, 1] (10) == M[1, 1] (10)? ———> NO CAMBIÓ
   |                                                  |
   |                                                  └——▶ 🔴 SE DESPORTA EL OBJETO 2
   |
[PASO 4] Celda M[1, 1] = 10
   |
   ├── ¿M[1, 1] (10) != M[0, 1] (0)?  ———> ¡SÍ CAMBIÓ!
   |                                                  |
   |                                                  ├——▶ 🟢 SE INCLUYE EL OBJETO 1
(Peso: 1, Valor: 10)
   |                                                  └——▶ Capacidad restante: 1 - 1 = 0
   |
[FIN] Capacidad = 0.
```

4. Cuadro Comparativo de la Solución Obtenida

Concepto	Resultado	Detalle del Cálculo
Salida directa del algoritmo	**18**	Devuelto por la instrucción M[longitud(O)-1, P]. Es un número entero que representa el valor máximo acumulable.
Objetos Seleccionados	**Objeto 1 y Objeto 4**	Deducidos mediante la reconstrucción inversa de la matriz.
Peso Total Consumido	**8 / 8**	Peso Objeto 1 (1) + Peso Objeto 4 (7) = 8 (Aprovechamiento del 100% de la capacidad).
Valor Total Ganado	**18**	Valor Objeto 1 (10) + Valor Objeto 4 (8) = 18.

5. Resumen de Complejidad y Comparativa

Complejidad Temporal: $\Theta(N \times P)$

Donde **N** es la cantidad de objetos y **P** es la capacidad entera de la mochila.

- **Tipo de Complejidad:** Seudopolinomial (depende del valor numérico de P, no de su tamaño en bits).
- **Comparación con Greedy:** Mientras que Greedy se resuelve en O(N log N) para la

mochila fraccionaria, en la mochila 0/1 solo la Programación Dinámica garantiza la solución óptima exacta.